CSCI 2800 — Computer Architecture and Operating Systems (CAOS)
Lab 7 (document version 1.0)

- This lab is due by the **end of your lab session** on Wednesday, October 21, 2026
- This lab is to be completed **individually**; do **not** share your code with anyone else
- Do **not** use AI to assist you; instead, use posted examples, the Discussion Forum, trial-and-error, and your own intelligence
- You **must** show your code and your solutions to a TA, CA, or mentor to receive credit for each checkpoint
- Note that all of your C code **must** successfully compile via `gcc` with no warning messages when the `-Wall` (i.e., warn all) compiler option is used; also use `-Werror`, which treats all warnings as critical errors

1. **Checkpoint 1:** To practice writing code in C using an "assembly language" mindset, we throw away all loop structures, restrict `if` conditions to checking a single variable without any operators, and simplify our statements to one calculation per statement. Each of these restrictions is described in more detail below.

   Instead of using loop keywords `for`, `do`, or `while`, you will need to add labels to your code and use `goto` statements. Loops typically have one or more conditions to check either before or after the loop code is executed. The same generally holds true for loops in assembly language. Labels are case sensitive, must start with a letter, and may contain letters, digits, and underscore (`_`) characters. Like naming variables, use meaningful and short label names. Note that you should try to use as few labels as possible.

   For your `if` statements, you are only allowed to treat a single variable as either *true* (i.e., non-zero) or *false* (i.e., `0`). No operators are allowed in the `if` condition, even the negation (`!`) operator. Further, compound conditions are not allowed anywhere, i.e., throw away the `&&` and `||` operators. Use the following code as an illustrative example of how to achieve `if` statement logic:

```
    ...
    int r1 = ( x > 512 );
    if ( r1 ) goto LABEL1;
    ...
    goto LABEL2; /* somewhere else... */
  LABEL1:
    printf( "x is greater than 512\n" );
    ...
```

   Finally, for any calculations you make, you are limited to one operator on the right-hand side of the assignment statement. As an example, compare the two snippets of code below:

```
float ftoc( float f )
{
#if 0
  /* simple solution in C */
  return ( f - 32 ) * 5.0 / 9.0;
#else
  /* assembly language mindset */
  float c;
  c = f;
  c = c - 32;     /* c -= 32;  */
  c = c * 5.0;    /* c *= 5.0; */
  c = c / 9.0;    /* c /= 9.0; */
  return c;
#endif
}
```

   Given the above constraints, re-implement Checkpoint 1 from Lab 04.

2. **Checkpoint 2:** If you have not yet done so, install the latest version of SPIM, which is the MIPS simulator we have been using in lecture.

   The SPIM URL is: `http://spimsimulator.sourceforge.net/`.

   Or on Linux, run the following command as `root`:

   ```
   bash# apt-get install spim
   ```

   After installing SPIM, copy-and-paste the MIPS code examples from lecture on Tuesday, October 20. Within the `spim` interface, enter the `?` command to see how to load a MIPS program, run the loaded program, step through the program one instruction at a time, etc. Be sure to also try using the `print` and `print_all_regs` commands.

   In general, refer to Appendix A of our textbook (available online at our library), as well as the "MIPS Reference Data Card," which is posted under Course Materials.

   Once you are fairly comfortable using `spim`, start a new MIPS program in `lab07-q2.s`. Following the style used in the posted MIPS code examples, translate the block of C code below into MIPS. Do not use any shortcuts, i.e., implement the exact steps shown.

   ```
   main()
   {
     int q2 = 13;
     q2 = 15 + q2 - 11;
     if ( q2 > -13 ) q2 -= 10;
     printf( "==> the answer is %d\n", q2 );
   }
   ```

   To accomplish this, associate variable `q2` with saved register `$s0`. In other words, register `$s0` will hold the value of `q2`.

   As a reminder, since register `$s0` is a *saved* register, its original value (i.e., before being set to 13 in your code) must be preserved on the stack before returning.

   To translate the `if` statement, use the `slt` instruction in conjunction with either the `beq` or `bne` instruction.

   Test your code with different initial values of `$s0`, including 2, -2, -13, -100, 1024, etc.

   Be fully prepared to explain what each MIPS instruction in your code does.

3. **Checkpoint 3: At the beginning of your Wednesday lab section, this lab checkpoint will be available that contains further instructions.** As a hint, keep working with MIPS code examples.